IDE & Debugging

Breakpoints, conditional breakpoints, logpoints; step in/out/over; watch variables; call stacks; hot code replace

---

Rahul Guha || SME || Java FS Training || HCL

4. Build Tools, IDE & Debugging

IDE & Debugging

1. Introduction to IDE and Debugging

IDE (Integrated Development Environment) is a software application that provides tools to write, compile, execute, test, and debug programs in a single environment.

Examples of Java IDEs:- Visual Studio Code - IntelliJ IDEA - Eclipse IDE - NetBeans

Debugging is the systematic process of identifying, analyzing, and fixing errors in a program.

Instead of repeatedly using System.out.println() to inspect a program, a debugger lets you pause execution, examine variables, trace method calls, and execute code one step at a time.

Why is debugging important?

- Identifies logical and runtime errors.
- Helps understand program execution.
- Reveals incorrect variable values.
- Tracks method calls and execution flow.
- Reduces the time required to troubleshoot code.

Debugging workflow diagram



IntelliJ IDEA: Complete Java Debugging Practical Guide

This practical is designed for a live classroom demonstration using IntelliJ IDEA. You will use one dummy Java program to demonstrate all seven debugging concepts:

- **Breakpoints and conditional**
- **Breakpoints**
- **Logpoints**
- **Step Into, Step Over, and Step Out**
- **Watch variables**
- **Call Stack**
- **Hot Code Replace (HotSwap)**

The goal is to open one project, run one program, and demonstrate each feature step by step without switching between multiple examples.

IntelliJ IDEA debugging workspace

```
package org.example;

public class DebuggingDemo {
    public static void main(String[] args) {
        int[] marks = {80, 90, 70, 60, 100};
        int bonus = 5;
        int total = 0;
        System.out.println("Student Marks Processing Started");
        for (int i = 0; i < marks.length; i++) {
            int adjusted = adjustMark(marks[i], bonus);
            total += adjusted;
            System.out.printf(
                    "Student %d: Raw=%d, Adjusted=%d, Total=%d%n",
                    i + 1, marks[i], adjusted, total
            );
        }
static int adjustMark(int mark, int bonus) {
        int adjusted = mark + bonus;
if (adjusted > 100) {
            adjusted = 100;
        }
        return adjusted;
    }
```

```
 double average = calculateAverage(total,
marks.length);

        String grade = gradeFor(average);

        System.out.println("----------------------------");
        System.out.println("Total Marks: " + total);
        System.out.println("Average: " + average);
        System.out.println("Grade: " + grade);
    }
static double calculateAverage(int total, int count) {
        return (double) total / count;
    }
    static String gradeFor(double average) {
        if (average >= 80) {
            return "A";
        } else if (average >= 60) {
            return "B";
        } else {
            return "C";
        }   }}
```

IntelliJ IDEA debugging workspace

```
        if (adjusted > 100) {
            adjusted = 100;
        }
        return adjusted;
    }
    static double calculateAverage(int total, int count) {
        return (double) total / count;
    }
    static String gradeFor(double average) {
        if (average >= 80) {
            return "A";
        } else if (average >= 60) {
            return "B";
        } else {
            return "C";
        } }
}
```

Expected output

```
Student Marks Processing Started
Student 1: Raw=80, Adjusted=85, Total=85
Student 2: Raw=90, Adjusted=95, Total=180
Student 3: Raw=70, Adjusted=75, Total=255
Student 4: Raw=60, Adjusted=65, Total=320
Student 5: Raw=100, Adjusted=100, Total=420
----------------------------
Total Marks: 420
Average: 84.0
Grade: A
```

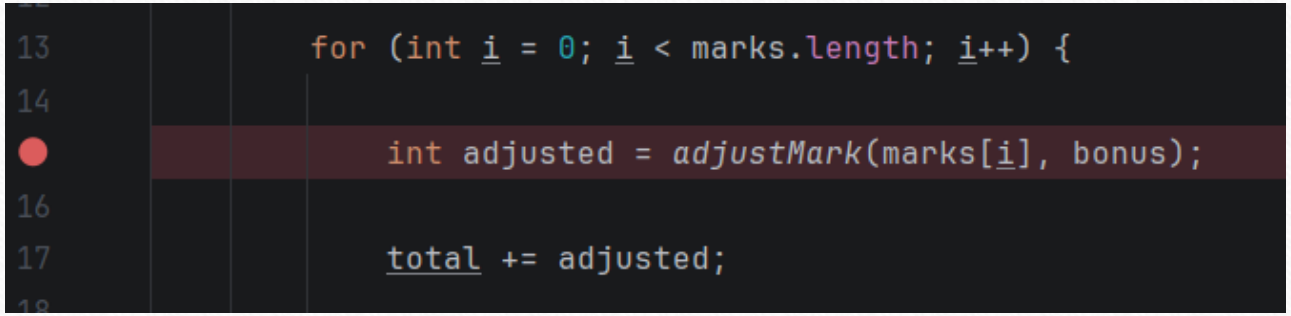
The last student's adjusted marks are capped at 100. This gives us a useful conditional branch to inspect.

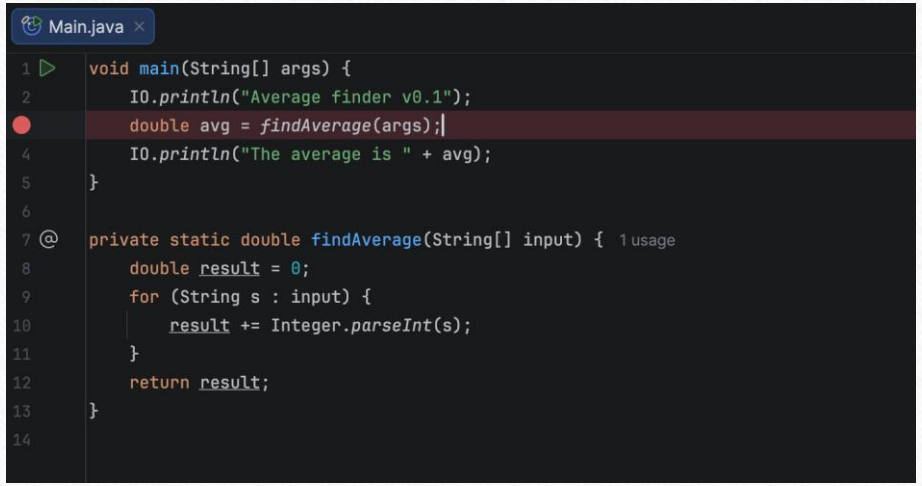
Demonstration 1 — Normal breakpoint

Objective: Pause execution at a specific line and inspect the current state. Step-by-step procedure

1. Open DebuggingDemo.java.
2. Locate this line inside the loop:
int adjusted = adjustMark(marks[i], bonus);
3. Click the gutter beside that line. A red breakpoint marker should appear.
4. Start the debugger using Shift + F9.The program pauses when it reaches the line.
5.Inspect the Variables panel.

What is a Breakpoint in Java?
A breakpoint is a marker you place on a line of code in an IDE, such as IntelliJ IDEA, to temporarily pause program execution during debugging.
It allows you to inspect variable values, check conditions, and understand how your program executes line by line.





Demonstration 1 — Normal breakpoint

For the first iteration, you should see values equivalent to:

Variable	Value
`i`	0
`marks[i]`	80
`bonus`	5
`total`	0

The statement has not executed yet, so adjusted has not received its value for this iteration.
What to explain to students: A breakpoint pauses the program before the selected statement executes, allowing us to inspect the state at that moment.

Objective: Pause execution at a specific line and inspect the current state. Step-by-step procedure
1. Open DebuggingDemo.java.
2. Locate this line inside the loop:
int adjusted = adjustMark(marks[i], bonus);
3. Click the gutter beside that line. A red breakpoint marker should appear.
4. Start the debugger using Shift + F9.The program pauses when it reaches the line.
5.Inspect the Variables panel.

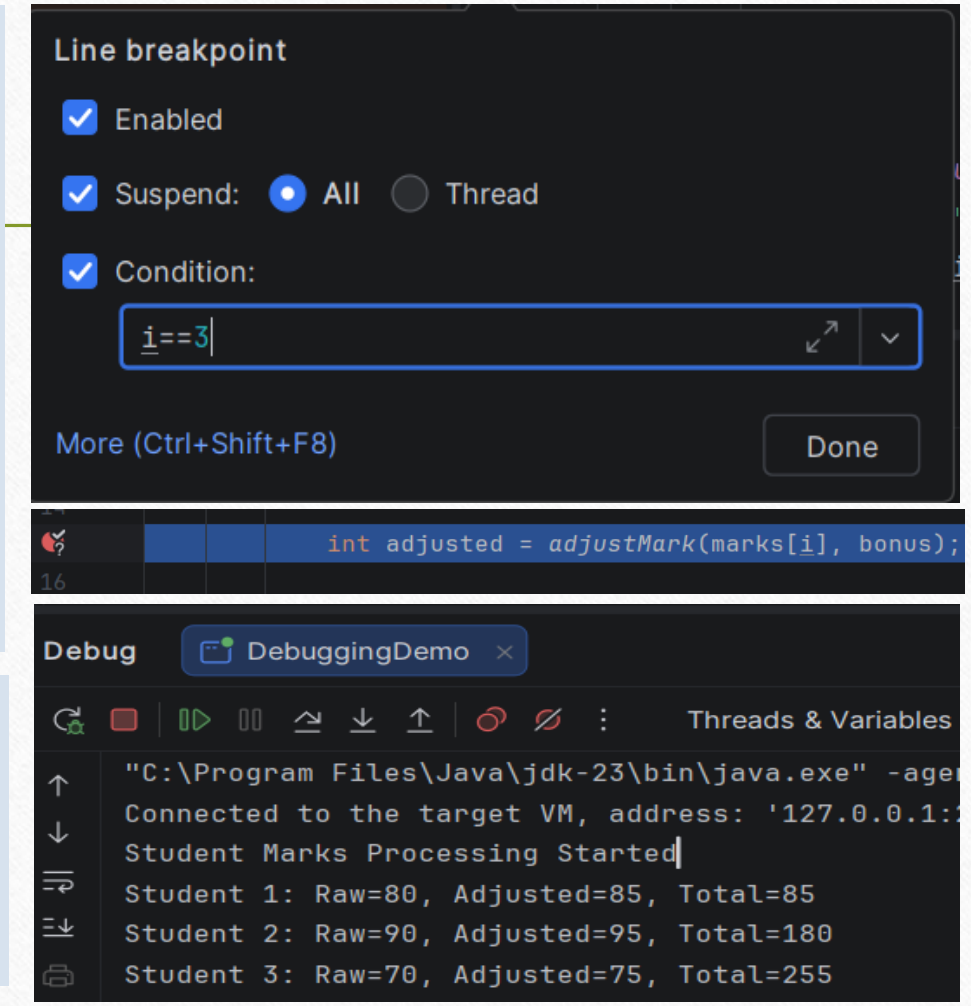
Demonstration 2 — Conditional breakpoint

Objective: Stop only when a particular condition is satisfied.
We will pause on the fourth student rather than stopping for all five students.

Steps:

1. Keep the breakpoint on int adjusted = adjustMark(marks[i], bonus);.
2. Right-click the breakpoint marker.
3. Configure the breakpoint condition.
4. Enter: i == 3
5. Apply the condition and restart the debugging session.
6. The debugger should pass the first three iterations and pause when i == 3.
7. Click Done or Close.

Remember that Java arrays are zero-indexed: i == 3 refers to the fourth student.
What to explain to students: A conditional breakpoint is useful when a bug appears only for a particular input, loop iteration, or state.



Demonstration 3 — Logpoints

Objective: Print diagnostic information without suspending execution.
IntelliJ IDEA supports non-suspending breakpoints with logging options.
Steps
1. Remove the conditional breakpoint or disable it.
2. Set a breakpoint beside this statement inside the loop: total += adjusted;
3. Right-click the breakpoint and open its settings.
4. Enable Log evaluated expression or the equivalent logging option available in your version.
5. Enter a logging expression such as:
"i=" + i + ", mark=" + marks[i]

- **+ ", adjusted=" + adjusted + ", total=" + total**

6. Disable Suspend for this breakpoint.
7. Apply the settings and resume or restart the program.

- **The exact labels may differ by IntelliJ IDEA version. If the breakpoint editor offers Log message to console, that is another way to log a custom message.**
  Example diagnostic output, assuming the logging point is on total += adjusted;:

```
"C:\Program Files\Java\jdk-23\bin\java.exe" "
Student Marks Processing Started
Student 1: Raw=80, Adjusted=85, Total=85
Student 2: Raw=90, Adjusted=95, Total=180
Student 3: Raw=70, Adjusted=75, Total=255
Student 4: Raw=60, Adjusted=65, Total=320
Student 5: Raw=100, Adjusted=100, Total=420
```

- The value of total in this logpoint is the value before the addition on that line. For example, the first log shows total=0 if the expression is evaluated before the statement executes.
- Logpoints help trace execution without repeatedly pausing the program or adding temporary print statements to the source code.

Demonstration 4 — Step Into, Step Over, and Step Out

Objective: Show how the debugger navigates between the main() method and other methods.

A. Step Into — F7

Set a breakpoint at:

int adjusted = adjustMark(marks[i], bonus);

Start debugging.

When execution pauses, press F7.

The debugger enters the adjustMark() method.

Inspect the method parameters:

mark = 80

bonus = 5

Step through the statements: int adjusted = mark + bonus;

The local variable becomes 85.

Continue through the if statement and return adjusted;.

B. Step Over — F8

1. Pause at a statement in main()

.2. Press F8.

3. The current statement executes.

4. If the statement calls a method, the debugger executes that method without entering its body.

For example, stepping over:

int adjusted = adjustMark(marks[i], bonus);

executes the method and returns to main(), where adjusted becomes 85.

C. Step Out — Shift + F8

1. Use Step Into to enter adjustMark().

2. Pause inside that method.

3. Press Shift + F8.

4. The debugger finishes the current method and returns to the caller.

This is useful when you have entered a method but no longer need to inspect its internal statements.

Demonstration 5 — Watch variables

Live demonstration summary

Action	What students should observe
Step Into	Execution enters `adjustMark()`
Step Over	Execution advances without entering the called method
Step Out	Execution returns from `adjustMark()` to `main()`

Note: If a method has already returned, Step Out cannot return from it again. Resume the program or select another active frame as appropriate.

Objective: Track values and evaluate expressions as the program runs.

Steps

1. Start debugging and pause inside the loop.
2. Open the Debug tool window.
3. Find the Watches section or the Watches tab.
4. Click + to add an expression.
5. Add these expressions one by one:

6. Use F8 to step through the loop and observe the changing values.

Values to demonstrate At the start of the first loop iteration, before adjustMark() has returned:

Watch expression	Value
`i`	0
`marks[i]`	80
`bonus`	5
`total`	0

After the statement `total += adjusted;` executes for the first iteration:

Watch expression	Value
`i`	0
`adjusted`	85
`total`	85
`total + adjusted`	170

The Watch panel evaluates expressions in the context of the currently selected stack frame. Variables that are out of scope may not be available.

Demonstration 6 — Call Stack

Objective: Show the method execution path and how the debugger reached the current line.

Steps

1. Set a breakpoint inside adjustMark():
int adjusted = mark + bonus;
2. Start debugging.
3. When execution pauses, open Frames in the Debug tool window.
4. Inspect the current method and its caller.
At this point, the stack should include:

```
Plain text

adjustMark()          <- Current method
main()                <- Caller
```

5. Resume the program with F9 until it reaches calculateAverage().
6. Inspect the stack again.

It should include:

```
Plain text

calculateAverage()    <- Current method
main()                <- Caller
```

7. Select a frame to inspect the local variables and source location for that method.

Teaching point: The Call Stack records the active chain of method invocations. It is especially useful for tracing exceptions and identifying the caller responsible for a problematic input.

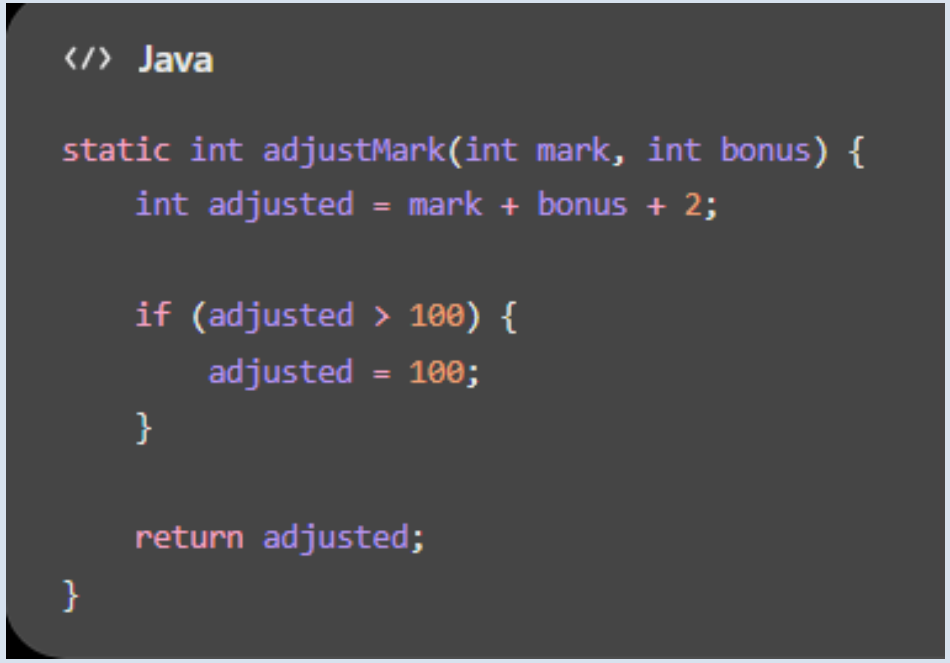
Demonstration 7 — Hot Code Replace (HotSwap)

Objective: Modify a method's implementation during an active debugging session and, when supported, apply the change without restarting the application.
We will change the bonus calculation.
Original method

```
static int adjustMark(int mark, int bonus) {
    int adjusted = mark + bonus;

    if (adjusted > 100) {
        adjusted = 100;
    }

    return adjusted;
}
```



Updated method
Change only the calculation line to: int adjusted = mark + bonus + 2;

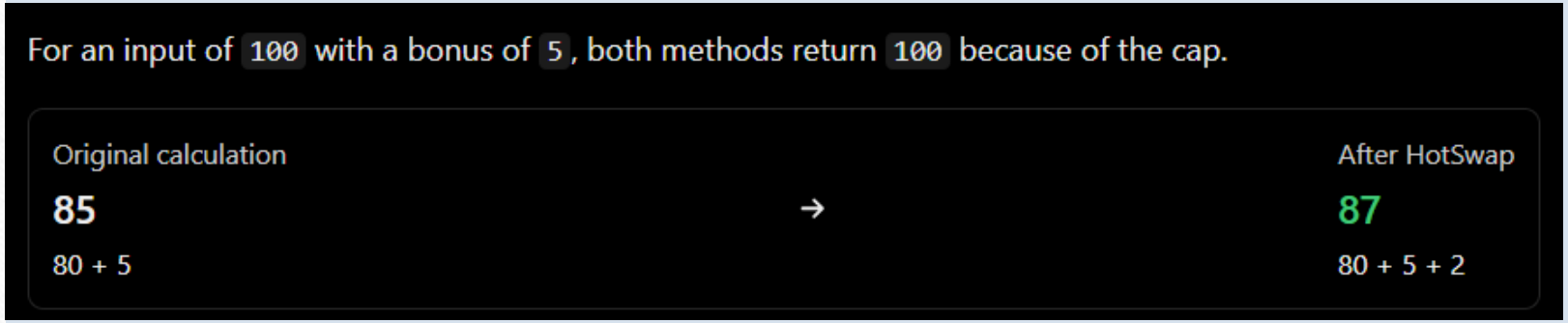
The updated method is:

Demonstration 6 — Call Stack

Step-by-step procedure

1. Start a debugging session with a breakpoint inside adjustMark().
2. Let the debugger pause.
3. Modify the calculation line in IntelliJ IDEA.
4. Compile the changed class using Build → Build Project (Ctrl + F9).
5. If IntelliJ IDEA offers to reload changed classes, accept the reload.
6. If no reload occurs automatically, use the available debugger reload action or check the debugger messages.
7. Resume execution and inspect a subsequent call to adjustMark().

For an input of 80 with a bonus of 5, the original calculation returns 85, while the updated calculation returns 87.



Important limitations: HotSwap depends on the JVM, debugger, and supported class changes. Traditional Java HotSwap generally supports method-body changes but not arbitrary structural changes such as adding fields or methods. If the reload fails, restart the debugging session to verify the updated code.

Things to Know before Lecture

Issue	What to do
Debugger does not start	Verify the project JDK and Java run configuration.
Breakpoint is not hit	Confirm that the line executes and the breakpoint is enabled.
Watch expression is unavailable	Pause execution and select the correct stack frame.
Logpoint pauses unexpectedly	Ensure **Suspend** is disabled.
Step Into skips the method	Confirm the current statement calls the method and check debugger stepping settings.
HotSwap fails	Check the reload message and class-change limitations; restart if necessary.

IDE & Debugging in IntelliJ IDEA

Debug Smarter
Build Better

Key Debugging Features & Their Real-World Use Cases

1 Breakpoints

Pause execution at a specific line to inspect variables and understand program flow.

```
4
   int sum = a + b;
6  System.out.println(sum);
7
```

Use case:
Debugging a billing calculation to check where the value goes wrong.

2 Conditional Breakpoints

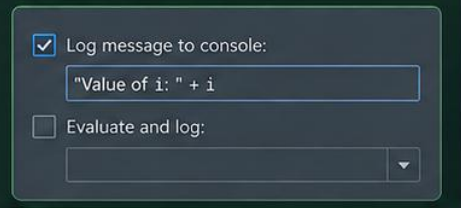
Pause execution only when a specific condition is true.



Use case:
Finding an incorrect record in a large loop (e.g., when i == 5).

3 Logpoints

Log variable values without pausing program execution.



Use case:
Monitoring transaction values in a running application without interrupting execution.

4 Step Into / Over / Out

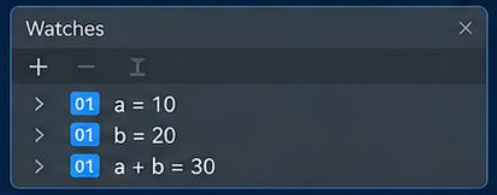
Trace statement execution and navigate between methods.



Use case:
Finding errors in nested method calls and understanding execution flow line by line.

5 Watch Variables

Monitor selected variables and evaluate expressions while debugging.



Use case:
Tracking marks, totals, and discounts during program execution.

6 Call Stack

Trace the sequence of methods that led to the current execution point.



Use case:
Identifying the source of an exception by seeing which methods were called.

7 Hot Code Replace (HotSwap)

Apply supported code changes during a debugging session without restarting the application.



Use case:
Quickly testing a modified calculation or method implementation without restarting the entire application.